

EmbedSim — Fachschale Tutorial

Buck Converter: Modelica → FMU → C → Cython → Python Block

Complete five-layer workflow — physics equations to Aurix TC3xx ISR

EmbedSim Framework · March 2026

1. What Is a Fachschale?

A **Fachschale** (domain layer) is EmbedSim's unit of reusable engineering IP. It is a self-contained Python package encapsulating a domain so it can be plugged into any simulation graph, validated against a high-fidelity FMU, and its C code generated for Aurix TC3xx.

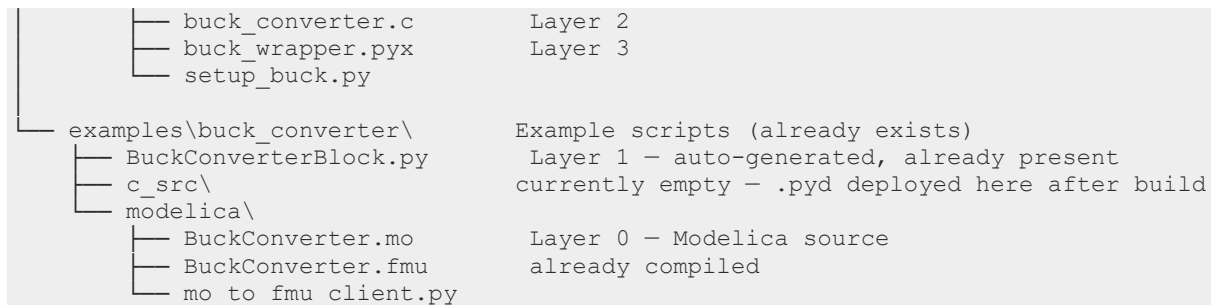
Layer	Purpose / File
0 — Modelica model	BuckConverter.mo — physics as equations; compiled to .fmu by OpenModelica
1 — FMU block	BuckConverterBlock.py — auto-generated by mo_to_fmu_client.py; wraps .fmu via FMI 2.0
2 — C source	buck_converter.c / .h — MISRA-C plant; identical binary on PC and Aurix
3 — Cython wrapper	buck_wrapper.pyx — zero-copy bridge to the C functions
4 — Python block	buck_block.py — VectorBlock subclass; compute() override; wires into the graph

💡 Layers 0–1 are for high-fidelity simulation. Layers 2–4 are the production path — the same C binary runs on PC (via Cython) and on Aurix (compiled by TASKING/HighTec).

2. Real Project Layout

Your project already has the reference Fachschale in `electrical_blocks\`. The Buck Converter Fachschale follows the same layout, creating a new package at project root:

```
C:\EmbedSimProject\
├── electrical_blocks\                REFERENCE Fachschale (existing)
│   ├── speed_pi_block.py            Layer 4 — overrides compute(t, dt, input_values)
│   ├── smc_block.py
│   └── modelica\
│       └── mo_to_fmu_client.py      FMU block generator (master copy)
├── c_src\
│   ├── speed_pi_controller.c / .h
│   ├── speed_pi_wrapper.pyx
│   └── setup_speed_pi.py
└── buck_converter\                 NEW Fachschale (this tutorial)
    ├── __init__.py
    ├── buck_block.py               Layer 4 — overrides compute(t, dt, input_values)
    └── c_src\
        ├── Sys_Types.h             copy from electrical_blocks\c_src\
        └── buck_converter.h         Layer 2
```



3. The compute() Pattern — How EmbedSim Blocks Work

This is the most important section to understand before writing `buck_block.py`. Every hand-written block in your project — `OpenLoopStartupBlock`, `MotorOutputBlock`, `NoisySensorBlock`, etc. — uses the same pattern:

3.1 What to override: `compute()` directly

Hand-written blocks override `compute(self, t, dt, input_values)`. This is the method `EmbedSim` calls each simulation step. The block reads from `input_values[port].value`, does its work, sets `self.output = VectorSignal(array, self.name)`, and returns it:

```
# Pattern used by ALL hand-written blocks in your project
class MyBlock(VectorBlock):

    def __init__(self, name):
        super().__init__(name)
        self.vector_size = 2          # declare output size

    def compute(self, t, dt, input_values=None):
        # Read from upstream block (port 0)
        u = input_values[0].value if input_values else np.zeros(2)

        # Your logic here
        y = u * 2.0

        # Must always set self.output and return it
        self.output = VectorSignal(np.array(y, dtype=np.float32), self.name)
        return self.output

    def reset(self):                 # optional - reset state for next run
        super().reset()
        self._state = 0.0
```

3.2 What NOT to use for hand-written blocks

The `compute_py()`, `compute_c()`, and `SimBlockBase` split is only for auto-generated `CodeGen` stubs emitted by `generate_pyx_stub()`. They add complexity without benefit for hand-written blocks where you control both paths. Use the clean pattern above.

Method	When to use
<code>compute(t, dt, input_values)</code>	Hand-written blocks — you always override this

compute_py() / compute_c()	Auto-generated stubs only — CodeGen output, not input
SimBlockBase	Only used in pmsm_foc_pwm_smc.py imports for the CodeGen markers; not for your block class

💡 BuckBlock uses compute() not compute_py(). The C backend (BuckWrapper) is selected inside compute() by checking self._wrap is not None — exactly as NoisySensorBlock checks its RNG, or MotorOutputBlock checks the input length.

4. Layer 0 — Modelica Model (BuckConverter.mo)

The file at `examples\buck_converter\modelica\BuckConverter.mo` is the authoritative physics source. Every C implementation must reproduce these equations exactly:

```

model BuckConverter
  "Simple average-value buck converter - CCM"

  parameter Real L      = 100e-6 "Inductance [H]";
  parameter Real C      = 100e-6 "Capacitance [F]";
  parameter Real R_load = 10      "Load resistance [Ohm]";
  parameter Real V_in   = 24      "Input voltage [V]";

  input Real duty        "PWM duty cycle [0-1]";

  output Real V_out      "Output capacitor voltage [V]";
  output Real I_L        "Inductor current [A]";
  output Real I_load     "Load current [A]";

protected
  Real switch_state;

equation
  switch_state = duty;

  L * der(I_L) = switch_state * V_in - V_out;
  C * der(V_out) = I_L - V_out / R_load;

  I_load = V_out / R_load; // algebraic - no der()

initial equation
  I_L = 0;
  V_out = 0;

end BuckConverter;

```

Equation	Physical meaning
$L \cdot \text{der}(I_L) = \text{duty} \cdot V_{\text{in}} - V_{\text{out}}$	Inductor dynamics — volt-second balance
$C \cdot \text{der}(V_{\text{out}}) = I_L - V_{\text{out}}/R$	Capacitor dynamics — current balance
$I_{\text{load}} = V_{\text{out}} / R_{\text{load}}$	Algebraic — computed directly, no integration

5. Layer 1 — FMU Block (BuckConverterBlock.py)

BuckConverterBlock.py already exists in `examples\buck_converter\`. It was generated by `mo_to_fmu_client.py` and wraps the DASSL-integrated BuckConverter.fmu. Regenerate it after changing the .mo:

```
> cd examples\buck_converter\modelica
> python mo_to_fmu_client.py BuckConverter.mo
```

The FMU block has four output variables: `V_out`, `I_L`, `I_load`, `switch_state`. Only the first three are used in validation against BuckBlock.

⚠ The FMU block uses the OpenModelica DASSL variable-step solver. It runs only on Win/Linux x64. The C layer below provides the Aurix-deployable equivalent.

6. Layer 2 — C Source (`buck_converter\c_src\`)

6.1 buck_converter.h

Copy `Sys_Types.h` from `electrical_blocks\c_src\` first. Then create the header — variable names match the Modelica model exactly:

```
/* buck_converter.h — MISRA-C:2012 | ASIL-B */
#ifndef BUCK_CONVERTER_H
#define BUCK_CONVERTER_H

#include "Sys_Types.h" /* real32_T = float */

/* State: I_L (inductor) + V_out (capacitor) */
typedef struct {
    real32_T iL; /* I_L [A] */
    real32_T vC; /* V_out [V] */
} BuckState_T;

/* Input per step */
typedef struct {
    real32_T duty; /* PWM duty cycle [0..1] */
    real32_T Vin; /* V_in [V] */
    real32_T R; /* R_load [Ohm] */
} BuckInput_T;

void Buck_Init (BuckState_T *s);
void Buck_Step (BuckState_T *s, const BuckInput_T *u,
               real32_T dt, real32_T L, real32_T C);
real32_T Buck_Vout (const BuckState_T *s);
real32_T Buck_IL (const BuckState_T *s);
real32_T Buck_Iload (const BuckState_T *s, real32_T R);

#endif /* BUCK_CONVERTER_H */
```

6.2 buck_converter.c

The `_deriv()` function is a direct one-to-one translation of the two Modelica `der()` lines. Compare each C line with the .mo equation section — they must be identical in structure:

```
/* buck_converter.c */
#include "buck_converter.h"
```

```

/* _deriv - translates Modelica equation section:
 *   L * der(I_L) = duty * V_in - V_out → *diL = (duty*Vin - vC) / L
 *   C * der(V_out) = I_L - V_out / R_load → *dvC = (iL - vC/R) / C */
static void _deriv(const BuckState_T *s, const BuckInput_T *u,
                  real32_T L, real32_T C,
                  real32_T *diL, real32_T *dvC)
{
    *diL = (u->duty * u->Vin - s->vC) / L;
    *dvC = (s->iL - s->vC / u->R) / C;
}

/* Buck_Init - Modelica: initial equation I_L = 0; V_out = 0 */
void Buck_Init(BuckState_T *s) { s->iL = 0.0f; s->vC = 0.0f; }

/* Buck_Step - RK4 fixed step */
void Buck_Step(BuckState_T *s, const BuckInput_T *u,
              real32_T dt, real32_T L, real32_T C)
{
    real32_T k1_iL, k1_vC, k2_iL, k2_vC, k3_iL, k3_vC, k4_iL, k4_vC;
    BuckState_T tmp;

    _deriv(s, u, L, C, &k1_iL, &k1_vC);
    tmp.iL = s->iL + 0.5f*dt*k1_iL; tmp.vC = s->vC + 0.5f*dt*k1_vC;
    _deriv(&tmp, u, L, C, &k2_iL, &k2_vC);
    tmp.iL = s->iL + 0.5f*dt*k2_iL; tmp.vC = s->vC + 0.5f*dt*k2_vC;
    _deriv(&tmp, u, L, C, &k3_iL, &k3_vC);
    tmp.iL = s->iL + dt*k3_iL; tmp.vC = s->vC + dt*k3_vC;
    _deriv(&tmp, u, L, C, &k4_iL, &k4_vC);

    s->iL += (dt/6.0f)*(k1_iL + 2.0f*k2_iL + 2.0f*k3_iL + k4_iL);
    s->vC += (dt/6.0f)*(k1_vC + 2.0f*k2_vC + 2.0f*k3_vC + k4_vC);
}

real32_T Buck_Vout (const BuckState_T *s) { return s->vC; }
real32_T Buck_IL   (const BuckState_T *s) { return s->iL; }
real32_T Buck_Iload(const BuckState_T *s, real32_T R) { return s->vC / R; }

```

⚠ Pass L and C as parameters — not globals. Two independent converter instances must never share a BuckState_T. This mirrors the pattern in speed_pi_controller.c.

7. Layer 3 — Cython Wrapper (buck_wrapper.pyx)

Mirrors speed_pi_wrapper.pyx. The private cdef BuckState_T _state is allocated on the Python object heap — no GIL overhead per step() call:

```

# buck_wrapper.pyx - cython: language_level=3, boundscheck=False

cdef extern from "buck_converter.h":
    ctypedef struct BuckState_T:
        float iL
        float vC
    ctypedef struct BuckInput_T:
        float duty; float Vin; float R
    void Buck_Init (BuckState_T *s)
    void Buck_Step (BuckState_T *s, const BuckInput_T *u, float dt, float L, float C)
    float Buck_Vout (const BuckState_T *s)
    float Buck_IL   (const BuckState_T *s)
    float Buck_Iload (const BuckState_T *s, float R)

cdef class BuckWrapper:

```

```

cdef BuckState_T _state    # private C struct – zero Python overhead
cdef float _L
cdef float _C

def __cinit__(self, float L=1e-4, float C=1e-4):
    self._L = L; self._C = C
    Buck_Init(&self._state)

def reset(self): Buck_Init(&self._state)

def step(self, float duty, float Vin, float R, float dt):
    """Returns (V_out, I_L, I_load) – matches BuckConverter.mo outputs."""
    cdef BuckInput_T u
    u.duty = duty; u.Vin = Vin; u.R = R
    Buck_Step(&self._state, &u, dt, self._L, self._C)
    return (Buck_Vout(&self._state),
            Buck_IL (&self._state),
            Buck_Iload(&self._state, R))

@property
def V_out(self): return Buck_Vout(&self._state)
@property
def I_L(self): return Buck_IL (&self._state)
def set_L(self, float v): self._L = v    # follows set_integrator() pattern
def set_C(self, float v): self._C = v

```

Build and deploy

```

> cd buck_converter\c_src
> python setup_buck.py build_ext --inplace
> copy buck_wrapper.cp312-win_amd64.pyd ..\
> copy buck_wrapper.cp312-win_amd64.pyd ..\..\examples\buck_converter\c_src\

```

8. Layer 4 — Python Block (buck_block.py)

8.1 Architecture decision: compute() not compute_py()

BuckBlock overrides `compute(self, t, dt, input_values)`. This is the same method signature used by every hand-written block in your project. Inside `compute()`, the C backend is selected by checking `self._wrap` is not `None`:

Condition	Path taken
<code>self._wrap</code> is not <code>None</code>	True: call <code>self._wrap.step(duty, Vin, R, dt)</code> – BuckWrapper (Cython)
<code>self._wrap</code> is <code>None</code>	False: call <code>self._py_rk4(duty, Vin, R)</code> – pure Python RK4

The wrapper is created in `reset()`. If the `.pyd` is not built, `self._wrap` stays `None` and the Python path runs silently. This matches the graceful fallback in `speed_pi_block.py` and `svpwm_block.py`.

8.2 Complete buck_block.py

```

# buck_converter/buck_block.py
from __future__ import annotations
import sys, numpy as np
from pathlib import Path

```

```

_HERE = Path(__file__).resolve().parent
_ROOT = _HERE.parent
for _p in [str(_ROOT), str(_HERE)]:
    if _p not in sys.path: sys.path.insert(0, _p)

from embedsim.core_blocks import VectorBlock, VectorSignal

sys.path.insert(0, str(_HERE / "c_src"))
try:
    from buck_wrapper import BuckWrapper as _BuckWrapper
    _C_AVAILABLE = True
except ImportError:
    _BuckWrapper = None; _C_AVAILABLE = False

class BuckBlock(VectorBlock):
    """
    VectorBlock wrapping the average-value Buck Converter.
    Physics from: examples/buck_converter/modelica/BuckConverter.mo

    Inputs  (port 0, size 3):  [duty, Vin, R_load]
    Outputs (size 3):  [V_out, I_L, I_load]

    compute() selects backend inside the same method – matching the pattern
    of all hand-written blocks in the project (NoisySensorBlock, etc.).
    NOT SimBlockBase – that is for auto-generated CodeGen stubs only.
    """
    C_INCLUDE = '"buck_converter.h"'
    C_SIGNATURE = ("void Buck_Step(BuckState_T *s, const BuckInput_T *u,"
                  " real32_T dt, real32_T L, real32_T C);")
    C_INPUT_MAP = [("cg_start", 0), ("cg_start", 1), ("cg_start", 2)]

    def __init__(self, name, L=100e-6, C=100e-6, R_load=10.0,
                 V_in=24.0, use_c_backend=True, **kwargs):
        super().__init__(name, **kwargs)
        self.vector_size = 3
        self.output_label = "[V_out, I_L, I_load]"
        self._L = float(L); self._C = float(C)
        self._R_default = float(R_load)
        self._Vin_default = float(V_in)
        self._dt = None
        self._wrap = None # set in reset() if .pyd available
        self._iL = 0.0 # Python fallback state
        self._vC = 0.0
        self._use_c = use_c_backend and _C_AVAILABLE

    def reset(self, dt=None):
        super().reset()
        if dt is not None: self._dt = float(dt)
        if self._use_c:
            self._wrap = _BuckWrapper(L=self._L, C=self._C) # C path
        else:
            self._iL = 0.0; self._vC = 0.0 # Python path

    def compute(self, t, dt, input_values=None):
        if self._dt is None: self._dt = float(dt)

        if input_values and input_values[0] is not None:
            u = input_values[0].value
            duty = float(np.clip(u[0], 0.0, 1.0))
            Vin = float(u[1]) if len(u) > 1 else self._Vin_default
            R = float(u[2]) if len(u) > 2 else self._R_default
        else:

```

```

        duty = 0.0; Vin = self._Vin_default; R = self._R_default

# — C backend —————
if self._wrap is not None:
    V_out, I_L, I_load = self._wrap.step(duty, Vin, R, self._dt)
# — Python fallback —————
else:
    V_out, I_L = self._py_rk4(duty, Vin, R)
    I_load = V_out / R

self.output = VectorSignal(
    np.array([V_out, I_L, I_load], dtype=np.float32), self.name)
return self.output

def _py_rk4(self, duty, Vin, R):
    """Pure-Python RK4 - numerically identical to Buck_Step in C."""
    dt, L, C = self._dt, self._L, self._C
    def _d(iL, vC):
        return (duty*Vin - vC)/L, (iL - vC/R)/C
    k1 = _d(self._iL, self._vC)
    k2 = _d(self._iL+.5*dt*k1[0], self._vC+.5*dt*k1[1])
    k3 = _d(self._iL+.5*dt*k2[0], self._vC+.5*dt*k2[1])
    k4 = _d(self._iL+ dt*k3[0], self._vC+ dt*k3[1])
    self._iL += (dt/6)*(k1[0]+2*k2[0]+2*k3[0]+k4[0])
    self._vC += (dt/6)*(k1[1]+2*k2[1]+2*k3[1]+k4[1])
    return self._vC, self._iL # V_out, I_L

def set_R_load(self, R): self._R_default = float(R)
def set_V_in (self, V): self._Vin_default = float(V)

```

9. Validation — FMU vs C

Run both backends on identical inputs and diff the outputs. This confirms the C equations are a faithful translation of the Modelica physics. Place the script in `examples\buck_converter\`:

```

# validate_c_vs_fmu.py - key section
def run_sim(plant, label):
    inp = BuckInputBlock() # outputs [duty=0.5, Vin=24.0, R=10.0]
    sink = VectorEnd("sink")
    inp >> plant >> sink
    sim = EmbedSim(sinks=[sink], T=5e-3, dt=1e-5, solver=ODESolver.RK4)
    sim.scope.add(plant, label=label)
    sim.run()
    # collect columns: V_out, I_L, I_load
    return np.column_stack([
        sim.scope.data.get(f"{label} [{i}]", []) for i in range(3)])

c_out = run_sim(BuckBlock("c", use_c_backend=True), "c")
fmu_out = run_sim(BuckConverterBlock("fmu",
    fmu_path="modelica/BuckConverter.fmu"), "fmu")

diff = np.abs(c_out - fmu_out[:len(c_out), :3])
print(f"Max V_out error: {diff[:,0].max()*1000:.3f} mV") # accept < 1 mV
print(f"Max I_L error: {diff[:,1].max()*1000:.3f} mA") # accept < 1 mA

```

Metric	Accept if	Notes
V_out error	< 1 mV	RK4 fixed-step vs DASSL variable-step solver tolerance

I_L error	< 1 mA	Same cause — integrator tolerance difference, not equation error
> 10 mV	FAIL	Check signs in <code>_deriv()</code> line-by-line against Modelica <code>der()</code> equations

10. Code Generation — Aurix TC3xx

Add `CodeGenStart` / `CodeGenEnd` markers around the chain exactly as in `pmsm_foc_pwm_smc.py`. `C_INPUT_MAP` maps the three input registers to duty, Vin, R_load:

```
# In your buck simulation script — follows pmsm_foc_pwm_smc.py pattern
from embedsim.code_generator import CodeGenStart, CodeGenEnd

cg_start = CodeGenStart("cg_start")
cg_end   = CodeGenEnd  ("cg_end")

plant.C_INPUT_MAP = [
    ("cg_start", 0),  # duty
    ("cg_start", 1),  # Vin
    ("cg_start", 2),  # R_load
]

cg_start >> plant >> cg_end

result = cg_end.generate_loop(
    cg_start   = cg_start,
    output_dir = _PROJECT_ROOT,  # writes into embedsim_gen/
    dt_hz      = 1.0 / DT,
    write_files= True,
)
```

Aurix ISR integration

```
/* isr_buck.c — Aurix TC3xx 100 kHz ISR */
#include "embedsim_loop.h"

void ISR_100kHz(void)
{
    motor_reg[0] = g_duty_setpoint;          /* duty from supervisor */
    motor_reg[1] = DSADC_GetResult(1) * SCALE; /* Vin from ADC          */
    motor_reg[2] = g_R_load_estimate;        /* R from param table */

    embedsim_loop_step(EMBEDSIM_DT);        /* runs Buck_Step      */

    GTM_ATOM_SetDuty(0, buck_out[0]);        /* V_out monitor       */
}
```

11. Completion Checklist

1. BuckConverter.mo — equations present, initial conditions $i_L=0$, $v_C=0$
2. BuckConverterBlock.py — generated from .mo, already in `examples\buck_converter\`
3. Sys_Types.h — copied from `electrical_blocks\c_src\`
4. buck_converter.h — `BuckState_T`, `BuckInput_T`, `Buck_Step` prototype
5. buck_converter.c — `_deriv()` matches Modelica equation section line-by-line

6. buck_wrapper.pyx — cdef class BuckWrapper, step() returns (V_out, I_L, I_load)
7. setup_buck.py — builds cleanly, .pyd produced
8. .pyd copied to buck_converter\ and examples\buck_converter\c_src\
9. buck_block.py — compute() overrides, C backend selected by self._wrap is not None
10. validate_c_vs_fmu.py — V_out < 1 mV, I_L < 1 mA
11. generate_loop() — embedsim_loop.c with no TODO comments

12. Troubleshooting

Symptom	Fix
V_out error > 10 mV vs FMU	Check sign in _deriv() — compare with Modelica der() equations one-to-one
ImportError: buck_wrapper	Rebuild setup_buck.py --inplace, copy .pyd to both target paths
V_out = 0 every step	Block not in sim.add() — reset() never called
self._wrap is None at runtime	_C_AVAILABLE is False — .pyd not found on sys.path in reset()
Algebraic loop error	Add VectorDelay on the V_out feedback path to a PI controller
TODO in embedsim_loop.c	C_INPUT_MAP indices wrong — check register assignments